

O protocolo HTTP

Descrición xeral

Protocolo de comunicacións estándar que comunica servidores, proxies e clientes. Permite a transferencia de documentos web, independentemente de cal sexa o cliente ou cal sexa o servidor.

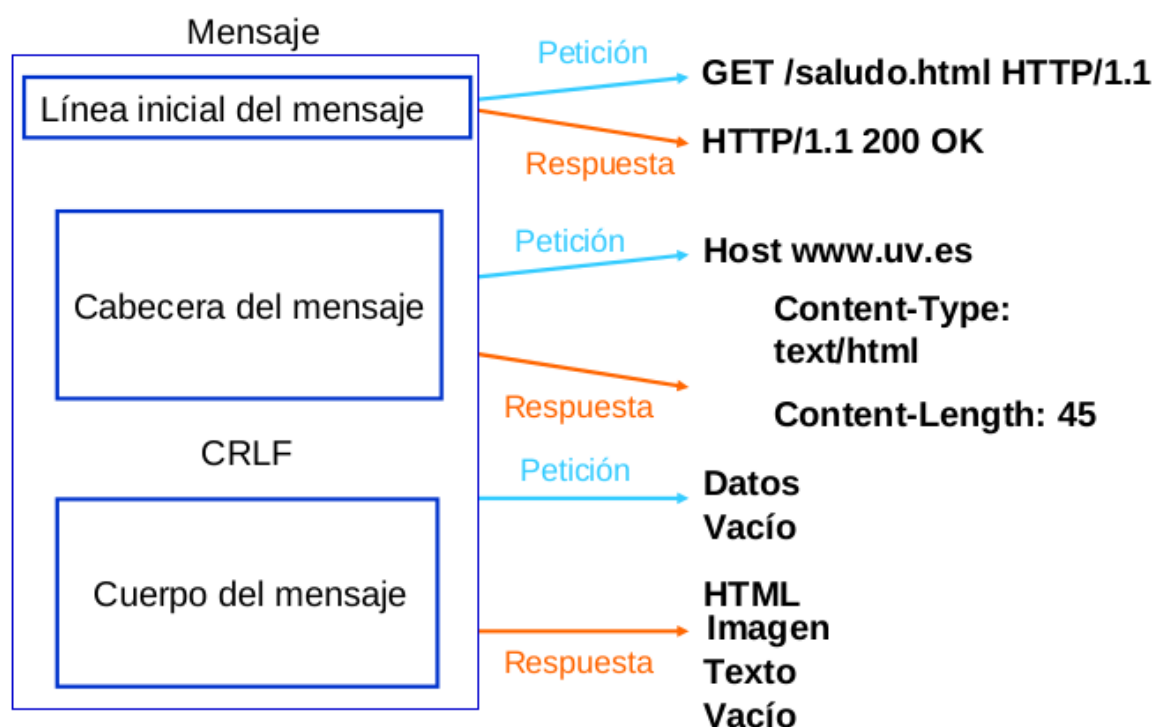
É un protocolo baseado no esquema de solicitude / resposta. O cliente fai unha solicitude e o servidor devolve unha resposta.

O protocolo HTTP está baseado en mensaxes de texto sen formato e é un protocolo sen estado.

Operación do protocolo

O usuario escribe un URL, indicando o protocolo, o servidor e o recurso que quere obter, o servidor procesa esta información e devolve unha mensaxe de resposta, normalmente co HTML da páxina que se amosará, ...

O contido da mensaxe segundo a solicitude ou a resposta pódese ver no seguinte esquema:



Métodos de envío dos datos

Ao facer unha solicitude, pódese usar un dos seguintes métodos:

- GET: solicitar un documento ao servidor. Os datos pódense enviar na URL.

- HEAD: semellante a GET, pero só pide cabeceiras HTTP.
- POST: envía datos ao servidor para o seu procesamento, similar a GET, pero tamén envía datos no corpo da mensaxe.
- PUT: almacena o documento enviado no corpo da mensaxe.
- DELETE: elimina o documento ao que se fai referencia na URL.
- ...

Código do estado

Cando o servidor devolve unha resposta, indícase un código de estado:

- | | |
|------------------------------------|----------------------------------|
| ▪ 1xx: Mensaje informativo. | ▪ 4xx: Error del cliente |
| ▪ 2xx: Exitos | ▪ 400 Bad Request |
| ▪ 200 OK | ▪ 401 Unauthorized |
| ▪ 201 Created | ▪ 403 Forbidden |
| ▪ 202 Accepted | ▪ 404 Not Found |
| ▪ 204 No Content | ▪ 5xx: Error del servidor |
| ▪ 3xx: Redirección | ▪ 500 Internal Server Error |
| ▪ 300 Multiple Choice | ▪ 501 Not Implemented |
| ▪ 301 Moved Permanently | ▪ 502 Bad Gateway |
| ▪ 302 Found | ▪ 503 Service Unavailable |
| ▪ 304 Not Modified | |

Cabeceiras

Tanto a solicitude como as respostas teñen unha serie de meta-información chamada cabeceiras, podemos indicar as máis importantes:

- Host: Nome e porto do servidor ao que se dirixe a solicitude.
- User-Agent: Identificación do programa do cliente.
- Server: indica o tipo de servidor HTTP empregado.
- Cache-control: o usa o servidor para indicarlle ao navegador que obxectos gardará na caché, por canto tempo, etc.,
- Content-type: Tipo MIME do recurso.
- Content-Encoding: indícase o tipo de codificación empregado na resposta.
- Expires: Indica unha data e hora despois das cales a resposta HTTP se considera obsoleta. Utilízase para xestionar a caché.
- Location: úsase para especificar unha nova localización en casos de redireccións.
- Set-Cookie: Solicita a creación dunha cookie no cliente.

Outras características

- **Cookies** : as cookies son información que o navegador garda na memoria ou no disco duro dentro dos ficheiros de texto, a petición do servidor.
- **Sesións** : HTTP é un protocolo sen manipulación de estados. As sesións permítenos definir estados, para iso o servidor almacenará a información necesaria para facer un seguimento da sesión.
- **Autenticación** : ás veces, debido a problemas de personalización ou políticas de restrición, as aplicacións web deben coñecer e verificar a identidade do usuario a través do nome de usuario e contrasinal.
- **Conexións persistentes** : permite transferir varias solicitudes e respostas usando a mesma conexión TCP.

Servidor Web Apache

Para instalar o servidor web Apache en sistemas GNU / Linux Debian / Ubuntu e derivados, executamos como administrador:

apt-get install apache2

Para controlar o servizo apache2 podemos usar

apache2ctl [-k start|restart|graceful|graceful-stop|stop]

A opción graceful é un reinicio suave, as solicitudes que se establecen rematan de publicarse e cando finaliza faise un reinicio do servidor.

Con esta ferramenta tamén podemos obter máis información do servidor:

- **apache2ctl -t** : Comproba a sintaxe do ficheiro de configuración.
- **apache2ctl -M** : Lista os módulos cargados.
- **apache2ctl -S** : Lista de sitios virtuais e opcións de configuración.
- **apache2ctl -V** : Lista de opcións de compilación

Obviamente o servidor está xestionado por Systemd iso, polo tanto, para controlar o inicio, reinicio e parada do servizo usaremos a seguinte instrución:

systemctl [start|stop|restart|reload|status] apache2.service

Estrutura dos ficheiros de configuración

O ficheiro de configuración principal de Apache2 é **/etc/apache2/apache2.conf**. Este ficheiro inclúe os ficheiros que forman parte da configuración de Apache2:

```
IncludeOptional mods-enabled/*.load
IncludeOptional mods-enabled/*.conf
...
Include ports.conf
...
IncludeOptional conf-enabled/*.conf
IncludeOptional sites-enabled/*.conf
```

- Os ficheiros engadidos gardados no directorio `mods-enabled` corresponden aos módulos activos.
- Os ficheiros engadidos ao directorio `sites-enabled` corresponden á configuración dos sitios virtuais activos.
- Desde o directorio `conf-enabled` engadimos ficheiros de configuración adicionais.
- Finalmente, no ficheiro `ports.conf` especificanse os portos de escoita do servidor.

Opcións de configuración para servidores virtuais

Por defecto, indícanse as opcións de configuración do directorio `/var/www` e de todos os seus subdirectorios, polo tanto as `DocumentRoot` dos hosts virtuais que se crean deben ser subdirectorios deste directorio, polo tanto atopamos no ficheiro `/etc/apache2/apache2.conf` o seguinte :

```
<Directory /var/www/>
    Options Indexes FollowSymLinks
    AllowOverride None
    Require all granted
</Directory>
```

Podemos indicar outro directorio como o directorio raíz do noso servidor virtual

```
#<Directory /srv/>
#    Options Indexes FollowSymLinks
#    AllowOverride None
#    Require all granted
#</Directory>
```

Engadir nova configuración

Se temos unha configuración adicional para o noso servidor, podemos gardala nun ficheiro (por exemplo `proba.conf`) dentro do directorio `/etc/apache2/conf-available`. Para engadir este ficheiro de configuración á configuración xeral do servidor empregamos a instrución:

```
# a2enconf proba
```

Esta afirmación crea unha ligazón simbólica no directorio `/etc/apache2/conf-enabled`. Para desactivar unha configuración empregamos:

```
# a2disconf proba
```

Variables de contorno Apache

O servidor HTTP Apache proporciona un mecanismo para almacenar información en variables especiais chamadas variables de contorno. Esta información pódese usar para controlar varias operacións como almacenar datos en ficheiros de rexistro (`log files`) ou controlar o acceso ao servidor. Podemos atopar estas variables definidas no ficheiro `/etc/apache2/envvars`.

Directivas

Estudiemos algunas directivas que podemos atopar en `/etc/apache2/apache2.conf`:

Directivas de control de la conexión

- [Timeout](#): define, en segundos, o tempo que o servidor agardará para recibir e transmitir durante a comunicación, por defecto é de 300 segundos, o que é apropiado para a maioría das situacións.
- [KeepAlive](#): Define se están activadas as conexións persistentes. Por defecto están activados.
- [MaxKeepAliveRequests](#): Establece o número máximo de solicitudes permitidas para cada conexión persistente. Por defecto está establecido en 100.
- [KeepAliveTimeout](#): Establece o número de segundos que o servidor esperará despois de atender unha solicitude antes de pechar a conexión. Por defecto 5 segundos

Otras directivas

- **User**: define o usuario que executa os procesos de Apache2.
- **Group**: define o grupo ao que corresponde o usuario.
- **LogLevel**: Controla o nivel de información que se garda nos ficheiros de rexistro ou logs.
- **LogFormat**: Controla o formato de información que gardará nos logs.
- **Directory** ou **DirectoryMatch**: Declara un contexto para un directorio e todos os seus directorios.
- **Files** ou **FilesMatch**: Declara un contexto para un conxunto de ficheiros.

Probando o noso servidor web

O servidor web Apache 2.4 está instalado por defecto coa configuración dun servidor virtual. A configuración deste sitio pódese atopar en:

`/etc/apache2/sites-available/000-default.conf`

E por defecto este sitio virtual está habilitado, polo que podemos verificar que hai unha ligazón simbólica a este ficheiro no directorio `/etc/apache2/sites-enabled`:

```
lrwxrwxrwx 1 root root 35 Oct 3 15:24 000-default.conf -> ../sites-available/000-default.conf
```

Unha das directivas máis importantes que atopamos no ficheiro de configuración é **DocumentRoot** onde se indica o directorio onde se almacenarán os ficheiros do noso sitio web, os ficheiros que serán servidos. Na configuración do host virtual por defecto o directorio é: `/var/www/html`

No ficheiro de configuración xeral `/etc/apache2/apache2.conf` atopamos as opcións de configuración do directorio pai do indicado na directiva **DocumentRoot**

(supoñemos que todos os hosts virtuais se almacenarán nos subdirectorios deste directorio):

```
...
<Directory /var/www/>
    Options Indexes FollowSymLinks
    AllowOverride None
    Require all granted
</Directory>
...
```

Polo tanto, podemos acceder desde un navegador á ip do noso servidor (tamén podemos usar un nome de servidor) e accederemos á páxina de benvida do servidor situada en:

`/var/www/html/index.html`

Por defecto gárdanse os erros do noso sitio virtual `/var/log/apache2/error.log` e gárdanse os accesos ao noso servidor `/var/log/apache2/access.log`.

Aloxamento virtual

O termo Aloxamento virtual refírese a executar máis dun sitio web (como `www.pagina1.com` e `www.pagina2.com`) nunha mesma máquina. Os sitios web virtuais poden "basearse en enderezos IP", o que significa que cada sitio web ten unha dirección IP diferente ou "basearse en nomes diferentes", o que significa que cunha única dirección IP funcionan sitios web con nomes diferentes. (De dominio) . Apache foi un dos primeiros servidores web en soportar aloxamento virtual baseado en enderezos IP.

Como vimos na unidade anterior, o servidor web Apache2 está instalado por defecto cun host virtual en `/etc/apache2/sites-available/000-default.conf`.

Cuxo contido podemos ver:

```
<VirtualHost *:80>
    #ServerName www.example.com
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html
    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
```

Onde atopamos os seguintes parámetros:

- [ServerName](#) : Nome polo cal se vai acceder ao host virtual.
- [ServerAdmin](#) : correo electrónico do responsable deste host virtual.
- [ServerAlias](#) : Outros nomes cos que se pode acceder ao sitio.
- [DocumentRoot](#) : directorio onde se gardan os ficheiros servidos neste host virtual.
- [ErrorLog](#) : arquivo onde se gardan os erros.
- [CustomLog](#) : ficheiro onde se gardan os accesos ao sitio.

Podemos activar ou desactivar os nosos hosts virtuais usando os comandos `a2ensite` e `a2dissite`.

Configuración de Aloxamento Virtual

Imos mostrar como podemos configurar diferentes sitios virtuais en Apache 2.4 (baseado no nome). O obxectivo é lanzar dous sitios web usando o mesmo servidor web apache. Débese ter en conta o seguinte:

- Cada sitio web terá nomes diferentes.
- Cada sitio web compartirá o mesmo enderezo IP e o mesmo porto (80).

Queremos construír dous sitios web no noso servidor web apache coas seguintes características:

- O nome de dominio do primeiro será `www.pagina1.org`, o seu directorio base será `/var/www/pagina1` e conterá unha páxina chamada `index.html`, onde verá unha mensaxe de benvida.
- O nome de dominio do primeiro será `www.pagina2.org`, o seu directorio base será `/var/www/pagina2` e conterá unha páxina chamada `index.html`, onde verá unha mensaxe de benvida.

Para obter estes dous sitios virtuais debes seguir os seguintes pasos:

1. Os ficheiros de configuración dos sitios web están no directorio `/etc/apache2/sites-available`; por defecto hai dous ficheiros, un chámase `000-default.conf` que é a configuración predeterminada do sitio web. Necesitamos dous novos ficheiros para configurar os dous sitios virtuais, para iso imos copiar o `000-default.conf` aos dous novos ficheiros:

```
cd /etc/apache2/sites-available
cp 000-default.conf pagina1.conf
cp 000-default.conf pagina2.conf
```

Deste xeito teremos un ficheiro chamado `pagina1.conf` para configurar o sitio web `www.pagina1.org` e outro chamado `pagina2.conf` para o sitio web `www.pagina2.org`.

2. Modificamos os ficheiros `pagina1.conf` e `pagina2.conf`, para indicar o nome que imos usar para acceder ao host virtual (`ServerName`) e ao directorio de traballo (`DocumentRoot`). Tamén poderíamos cambiar o nome do ficheiro onde se gardan os rexistros.
3. Non basta con crear os ficheiros de configuración para cada sitio web, é necesario crear unha ligazón simbólica a estes ficheiros dentro do directorio `/etc/apache2/sites-enabled`.

A creación das ligazóns simbólicas pódese facer coa instrución `a2ensite nome_fichero_configuracion`, para desactivar o sitio temos que eliminar a ligazón simbólica ou usar a instrución `a2dissite nome_fichero_configuracion`.

a2ensite pagina1

a2ensite pagina2

4. Creamos os directorios e arquivos necesarios `index.html` en `/var/www` e reinicie o servizo. Lembre que os ficheiros servidos deben ser propiedade do usuario e do grupo empregados por Apache, é dicir, usuario `www-data` e grupo `www-data`.

```
# chown -R www-data:www-data /var/www/pagina1
# chown -R www-data:www-data /var/www/pagina2
```

5. Para rematar, todo o que temos que facer é cambiar o ficheiro `hosts` nos clientes e colocar dúas novas liñas onde se faga a conversión entre os dous nomes de dominio e a dirección IP do servidor ou ben dar de alta os nomes dos servidores no servidor DNS

Configuración de acceso a servidores virtuais

Configuración de portos de escoita

Para determinar os portos de escoita do servidor web empregamos a directiva **Listen** que podemos modificar no ficheiro `/etc/apache2/ports.conf`.

Como funciona nos hosts virtuais

`Listen` só lle di ao servidor principal en que enderezos e portos pode escoitar. Se non se utilizan directivas `<VirtualHost>`, o servidor comportase igual con todas as peticións que se aceptan. Non obstante, `<VirtualHost>` pódese usar para especificar un comportamento diferente nun ou máis enderezos e portos. Para implementar un host virtual, primeiro ten que indicarlle ao servidor que escoite nos enderezos e portos a usar. Posteriormente, débese crear unha sección `<VirtualHost>` nun enderezo e porto específicos para determinar o comportamento dese host virtual.

Por defecto, os hosts virtuais que definimos responden dende calquera IP do porto 80, no ficheiro `/etc/apache2/sites-available/000-default.conf` atopamos:

```
<VirtualHost *:80>
```

Exemplo: servidor virtual baseado en IP

Neste caso, a nosa máquina debe ter varias IP configuradas (por exemplo con dúas interfaces ou varias ips nunha mesma interface), para cada IP haberá un host virtual.

```
<VirtualHost 192.168.56.3:80>
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/externa
    ErrorLog ${APACHE_LOG_DIR}/error_externa.log
    CustomLog ${APACHE_LOG_DIR}/access_externa.log combined
</VirtualHost>
```

```
<VirtualHost 172.22.0.1:80>
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/interna
```



```
ErrorLog ${APACHE_LOG_DIR}/error_interna.log
CustomLog ${APACHE_LOG_DIR}/access_interna.log combined
</VirtualHost>
```

Exemplo: publique o mesmo contido en varias IP

Supoñemos que o noso servidor ten dúas interfaces de rede (unha interface interna (intranet) e unha interface externa (internet)), queremos que responda aos dous enderezos:

```
<VirtualHost 192.168.56.3 172.22.0.1>
    DocumentRoot /var/www/externa
    ServerName servidor.example.com
    ServerAlias servidor
    ...
</VirtualHost>
```

Exemplo: servindo diferentes sitios en diferentes portos

Esta vez definimos dous portos de escoita no ficheiro `/etc/apache2/ports.conf`:

```
Listen 80
Listen 8080
```

E a configuración dos hosts virtuais podería ser a seguinte:

```
<VirtualHost *:80>
    ServerName servidor.example.com
    DocumentRoot /var/www/externa
</VirtualHost>

<VirtualHost *:8080>
    ServerName servidor.example.com
    DocumentRoot /var/www/interna
</VirtualHost>
```

Mapeo de URL

Opcións de directorio

Cando indicamos a configuración dun servidor apache, por exemplo coa directiva [Directory](#), podemos indicar algunhas opcións coa directiva [Options](#). Algunhas das opcións que podemos indicar son as seguintes:

- All: Todas as opcións agás MultiViews.
- FollowSymLinks: Pódense seguir ligazóns simbólicas.
- Indexes: Cando accedemos ao directorio e non se atopa un ficheiro predeterminado (indicado na directiva `DirectoryIndex` do módulo `mod_dir`), por exemplo `index.html`, móstrase a lista de ficheiros (faino o módulo `mod_autoindex`).
- MultiViews: Permite a negociación de contidos a través do módulo `mod_negotiation`.
- SymLinksIfOwnerMatch: Pódense seguir as ligazóns simbólicas só cando o ficheiro de destino é do mesmo propietario que a ligazón simbólica.
- ExecCGI: Permite executar o script CGI usando o módulo `mod_cgi`.

Podemos activar ou desactivar unha opción en referencia á configuración dun directorio pai mediante o signo +ou -.

Exemplo

No ficheiro `/etc/apache2/apache2.conf` atopamos o seguinte código:

```
<Directory /var/www/>
    Options Indexes FollowSymLinks
    ...
```

Entón podes cambiar as opcións do host virtual `pagina1`, incluído no seu ficheiro de configuración:

```
<Directory /var/www/pagina1>
    Options -Indexes +MultiViews
    ...
```

Traballando con alias

A directiva [Alias](#) permite ao servidor servir ficheiros desde calquera lugar do sistema de ficheiros aínda que estea fóra do directorio indicado no `DocumentRoot`.

Por exemplo, se poño este alias no ficheiro de configuración de `pagina1`:

```
Alias "/image" "/ftp/pub/image"
```

Podo acceder, por exemplo, a unha imaxe coa url `www.pagina1.org/image/logo.jpg`.

Non basta con poñer a directiva `Alias`, tamén é necesario dar permiso para acceder ao directorio, polo tanto teremos que poñer:

```
Alias "/image" "/ftp/pub/image"
<Directory "/ftp/pub/image">
    Require all granted
</Directory>
```

Podemos usar a directiva `AliasMatch` dun xeito similar ao `Alias` empregando expresións regulares para determinar o URL ao que se accede. Por exemplo:

```
AliasMatch "^/image/(.*)$" "/ftp/pub/image/$1"
```

Ollo! Para que só funcione para ese virtualhost hai que poñelo dentro. Se o poñemos fóra será unha directiva válida para todos os sites.

Negociación de contidos

Apache pode escoller a mellor representación dun recurso en función das preferencias proporcionadas polo navegador para diferentes tipos de soportes, idiomas, conxuntos de caracteres e codificación. Esta funcionalidade chámase **negociación de contido**. Un recurso pode estar dispoñible en diferentes representacións. Por exemplo, pode estar dispoñible en diferentes idiomas, é posible que o servidor faga automaticamente unha selección da páxina que se vai publicar. Isto funciona porque os navegadores poden enviar como parte da súa solicitude información sobre a representación que prefiren. Por exemplo, un navegador pode indicar que prefire ver información en galego e, se non é posible, en castelán. Os navegadores indican as súas preferencias mediante cabeceiras na solicitude. Para solicitar representacións só en español, o navegador podería enviar algo así:

```
Accept-Language: es
```

Para levar a cabo esta funcionalidade, Apache2 usa o módulo `negotiation_module` que está habilitado por defecto.

Configuración do módulo de negociación de contido con Multiviews

Queremos que ao acceder ao ULR `www.pagina1.org/internacional` se mostre un `index.html` coa linguaxe axeitada segundo a cabeceira `Accept-Language` enviada polo cliente.

O primeiro que temos que facer é crear varios ficheiros `index.html` cos diferentes idiomas que ofrece o servidor:

```
# ls /var/www/html/internacional
index.html.en  index.html.es
```

Creamos dous ficheiros: `index.html.en` para o idioma inglés e `index.html.es` para o español.

A continuación debemos activar a opción `Multiviews` para o directorio co que estamos a traballar, polo tanto, no ficheiro de configuración do host virtual

`/etc/apache2/sites-available/pagina1.conf` creamos unha sección

`Directory:`

```
...
<Directory /var/www/html/internacional>
    Options +Multiviews
</Directory>
...
```

Agora só temos que configurar o idioma no navegador e acceder á URL e podemos comprobar como se serven as diferentes páxinas segundo o idioma seleccionado.

Redireccións

A directiva `redirect` úsase para pedir ao cliente que faga outra solicitude a un URL diferente. Normalmente usámolo cando o recurso ao que queremos acceder cambiou de localización.

Podemos crear redireccións de dous tipos:

- **Permanente** : prodúcese cando o recurso sobre o que se realiza a solicitude foi "movido permanentemente" a un enderezo diferente, é dicir, a outro URL. Devólvese o código de estado 301. É o que debemos facer cando queremos cambiar a URL dun recurso para que os motores de busca, por exemplo cando cambiamos de dominio, nos siga gardando a posición que tiña a nosa páxina.
- **Temporal** : prodúcese cando se atopou o recurso sobre o que se realizou a solicitude pero reside temporalmente noutro enderezo diferente, é dicir, noutra URL. Devólvese un código de estado 302. Esta é a opción predeterminada.

Exemplos de redireccións temporais

```
Redirect "/service" "http://www.pagina.com/service"
Redirect "/one" "/two"
```

Como podemos ver, podemos redirixir a URL doutro servidor ou do mesmo servidor.

Exemplos de redireccións permanentes

```
Redirect permanent "/one" "http://www.pagina2.com/two"
Redirect 301 "/outro" "/outra"
```

Páxinas de erro personalizadas

Apache ofrece a posibilidade de que os administradores poidan configurar as respostas que o servidor Apache mostra cando se producen algúns erros ou problemas. Pódese facer que o servidor siga un dos seguintes comportamentos:

1. Amosa un texto diferente en lugar das mensaxes que aparecen por defecto.
2. Redirixir a solicitude a un URL local.
3. Redirixir a solicitude a un URL externo.

A directiva `ErrorDocument` permíteme configurar unha páxina de erro personalizada para cada tipo de erro (código de estado). Por exemplo:

```
ErrorDocument 403 "Sorry can't allow you access today"  
ErrorDocument 404 /error/404.html  
ErrorDocument 500 http://www.pagina2.org/error.html
```

Podemos usar a directiva `ErrorDocument` en diferentes áreas da nosa configuración, por exemplo, se a colocamos dentro dun host virtual, as páxinas de erro personalizadas só se verán nese host virtual.

Se queremos configurar páxinas de erro personalizadas podemos facelo nun arquivo de configuración: `/etc/apache2/conf-available/localized-error-pages.conf`. Este ficheiro de configuración está activo, podemos ver a ligazón simbólica que existe no directorio `/etc/apache2/conf-enabled`.

Control de acceso: Autenticación e autorización

Control de acceso. Autorización

O control de acceso refírese a todos os medios que proporcionan un xeito de controlar o acceso a calquera recurso. A directiva `Require` ofrece diversos xeitos de permitir ou denegar o acceso aos recursos. Se poñemos varias directivas `Require` dentro dunha mesma sección, sen estar dentro dun contedor de autorización, permitirase o acceso de cumprirse algunha delas. Os contedores son: `RequireAll`, `RequireAny` e `RequireNone`.

```
<RequireAll>
    Require...
</RequireAll>
```

Directiva `Require`

Podemos controlar o acceso a calquera recurso ou conxunto de recursos, por exemplo usando unha directiva `Directory` con `Require` usando algunhas destas opcións:

- `Require all granted`: Permítese o acceso sen condicións.
- `Require all denied`: O acceso é denegado incondicionalmente.
- `Require user userid [userid] ...`: O acceso só se permite se se autenticaron os usuarios indicados.
- `Require group group-name [group-name] ...`: Permítese o acceso só a grupos de usuarios especificados.
- `Require valid-user`: Permítese o acceso a usuarios válidos.
- `Require ip 10 172.20 192.168.2`: Permítese o acceso se se fai dende o conxunto de enderezos especificados.
- `Require host dominio`: Permítese o acceso se se fai desde o dominio especificado.
- `Require local`: Permítese o acceso desde `localhost`.

Podes usar o operador `not` para indicar denegación, por exemplo:

```
Require not ip 10.0
```

Control de acceso en Apache 2.2

En versións anteriores de Apache outras directivas son usados para control de acceso: `Allow`, `Deny` e `Order` son obsoletos e eliminaranse en versións futuras.

Exemplo 1

Por exemplo, na versión Apache 2.2 podemos atopar esta configuración:

```
<Directory "/var/www">
    Order allow,deny
    Allow from all
</Directory>
```

- Este é un modo **Denegar por defecto** . Onde opcionalmente se lle dará unha lista de regras de *Permitir* .
- *Establécense* as regras *Permitir* e o acceso disposto debe coincidir polo menos cunha regra.
- Se alguén recibe o permiso dunha das regras *Permitir* , pode negalo cunha regra *Denegar* .

En Apache 2.4 sería:

```
<Directory "/var/www">
    Require all granted
</Directory>
```

Outro exemplo, por exemplo, para non permitir o acceso ao ficheiro `.htaccess`, podémolo atopar en Apache 2.2:

```
<FilesMatch "^\.ht">
    Order deny,allow
    Deny from all
</FilesMatch>
```

- Este é un modo de **permiso predeterminado** . Onde opcionalmente darache unha lista de regras de *denegación* .
- A continuación, compróbanse as regras *Denegar* para rexeitar as solicitudes en función destas regras.
- Se alguén é rexeitado por unha das regras de *denegación* , pode devolvelo cunha regra *Permitir* .

En Apache 2.4 sería:

```
<FilesMatch "^\.ht">
    Require all denied
</FilesMatch>
```

Exemplo 2

En Apache 2.2 poderíamos ter:

```
Order Deny,Allow
Deny from all
Allow from example.org
```

En Apache 2.4 teremos:

```
Require host example.org
```

Autenticación básica

O servidor web Apache pode ir acompañado de diferentes módulos para proporcionar diferentes modelos de autenticación. A primeira forma que veremos é a máis sinxela. Usamos para isto o módulo básico de autenticación que se instala por defecto con calquera Apache `mod_auth_basic`. A configuración que temos que engadir no ficheiro de definición do servidor virtual a protexer podería ser algo así:

```
<Directory "/var/www/pagina1/privado">
    AuthUserFile "/etc/apache2/claves/passwd.txt"
    AuthName "Palabra de paso"
    AuthType Basic
    Require valid-user
</Directory>
```

O método de autenticación básico especificase na directiva `AuthType` .

- En `Directory` escribimos o directorio a protexer, que pode ser o raíz do noso Virtual Host ou un subdirectorio.
- En `AuthUserFile` poñemos o ficheiro que gardará a información de usuario e contrasinal que debería estar, como neste exemplo, nun directorio que non se pode visitar dende o noso Apache. Agora imos discutir como xeralo.
- Finalmente, `AuthName` personalizamos a mensaxe que aparecerá na xanela do navegador que nos pedirá o contrasinal.
- Para controlar o control de acceso, é dicir, os usuarios que teñen permiso para usar o recurso, empregamos as seguintes directrices: `AuthGroupFile`, `Require user`, `Require group`.

A utilidade que xera o ficheiro de contrasinal é `htpasswd`. A súa sintaxe é moi sinxela. Para engadir un novo usuario ao ficheiro funcionamos así:

```
$ htpasswd /etc/apache2/claves/passwd.txt carolina
New password:
Re-type new password:
Adding password for user carolina
```

Para crear o ficheiro de contrasinal coa introdución do primeiro usuario temos que engadir a opción `-c`(crear) ao comando anterior. Se por erro seguimos empregándoo ao incorporar novos usuarios, eliminaremos todos os anteriores, así que ten coidado con isto. Os contrasinais, como podemos ver a continuación, non están en claro. O que se almacena é o resultado de aplicar unha *función hash* :

```
josemaria:r0UetcAKYaliE
carolina:hm06V4bM8KLdw
alberto:9RjyKKYK.xyhk
```

Para denegar o acceso a un usuario, é suficiente con que eliminemos a liña correspondente a el. Non precisamos pedirlle a Apache que volva a ler a súa configuración cada vez que facemos cambios neste ficheiro de contrasinal.

Se queres permitir un grupo de usuarios, terás que crear un ficheiro de grupo que asocie os nomes do grupo co nome de usuario para permitirlles o acceso. O formato deste

ficheiro é bastante sinxelo e podes crealo co teu editor de texto favorito. O contido do ficheiro terá o seguinte aspecto:

```
NombreGrupo: usuario1 usuario2 usuario3
```

A directiva que teríamos que empregar para iniciar o ficheiro de grupo sería [AuthGroupFile](#). E para permitir o acceso aos grupos usaríamos:

```
Require group NombreGrupo
```

A principal vantaxe deste método é a súa sinxeleza. Os seus inconvenientes: o inconveniente de delegar a xeración de novos usuarios en alguén que non sexa un administrador do sistema ou de facer un front-end para que sexa o propio usuario o que cambie o seu contrasinal. E, por suposto, que eses contrasinais viaxan claramente pola rede. Se queremos evitar isto último podemos configurar Apache2 con SSL.

Implementación de políticas de autenticación e acceso

Políticas de acceso en Apache 2.2

Como vimos anteriormente, o control de acceso nas versións anteriores do apache2 se fixo cos Order, Allowe directivas Deny. Ademais, tivemos outra política [Satisfy](#) (**Nota: Esta directiva non existe no Apache 2.4**) que nos permitiu a comportarse de control como debe ser o servidor cando temos varias instrucións de control de acceso (allow, deny, require). por aquí:

- Satisfy All: Tiveron que cumprirse todas as condicións para acceder.
- Satisfy Any: Bastaba con que se cumprise unha das condicións.

Exemplo:

```
<Directory /dashboard>
    Order deny,allow
    Deny from all
    Allow from 10.1
    Require group admins
    Satisfy any
</Directory>
```

O uso destas directivas dificultou a elaboración de políticas de acceso complexas.

Políticas de acceso en Apache 2.4

Na nova versión, o control de acceso determínase coa directiva [Require](#) e as políticas de acceso pódense indicar mediante as directivas:

- [RequireAll](#): Todas as condicións dentro do bloque deben cumprirse para acceder.

- **RequireAny**: Debe cumprirse polo menos unha das condicións do bloque.
- **RequireNone**: Non se debe cumprir ningunha das condicións para permitir o acceso.

O exemplo anterior sería:

```
<Directory /dashboard>
    <RequireAny>
        Require ip 10.1
        Require group admins
    </RequireAny>
</Directory>
```

Exemplo

En Apache 2.2 poderíamos ter:

```
Order Allow,Deny
Allow from all
Deny from 212.100.100.100
```

En Apache 2.4 poderíamos indicalo de dous xeitos diferentes:

```
<RequireNone>
    Require ip 212.100.100.100
</RequireNone>
```

Ou tamén:

```
<RequireAll>
    Require all granted
    Require not ip 212.100.100.100
</RequireAll>
```

Exemplo complexo

Podemos crear varios bloques como vemos no seguinte exemplo:

```
<RequireAny>
    <RequireAll>
        Require user root
        Require ip 123.123.123.123
    </RequireAll>
    <RequireAll>
        <RequireAny>
            Require group sysadmins
            Require group useraccounts
            Require user anthony
        </RequireAny>
        <RequireNone>
            Require group restrictedadmin
            Require host bad.host.com
        </RequireNone>
    </RequireAll>
</RequireAny>
```

Usando módulos en Apache 2.4

Un dos aspectos característicos do servidor HTTP Apache é a súa modularidade, Apache ten unha serie de funcións adicionais que se sempre se inclúsen faríano demasiado grande e pesado. Pola contra, Apache compílase de xeito modular e só se cargan na memoria os módulos necesarios en cada caso.

Os módulos gárdanse na configuración apache2 en dous directorios:

- `/etc/apache2/mods-available/`: Directorio que contén os módulos dispoñibles na instalación actual.
- `/etc/apache2/mods-enabled/`: Directorio que inclúe a través de ligazóns simbólicas ao directorio anterior, os módulos que se cargarán na memoria a próxima vez que se inicie Apache.

Módulos Apache

Os módulos Apache pódense atopar de dous xeitos, compilados dentro do executable apache2 ou compilados individualmente como unha biblioteca de ligazóns dinámicas (con extensión `.so`). Para saber que módulos inclúe o executable da nosa instalación de apache, podemos empregar a seguinte instrución:

```
# apache2 -l
Compiled in modules:
  core.c
  mod_so.c
  mod_watchdog.c
  http_core.c
  mod_log_config.c
  mod_logio.c
  mod_version.c
  mod_unixd.c
```

O resto dos módulos dispoñibles para cargar en tempo de execución están no directorio `/usr/lib/apache2/modules/`:

```
# ls /usr/lib/apache2/modules/

httpd.exp          mod_dav.so         mod_proxy_fcgi.so
mod_access_compat.so  mod_dbd.so        mod_proxy_fdpass.so
mod_actions.so      mod_deflate.so     mod_proxy_ftp.so
...
```

Pode parecer moito, pero só son os módulos da instalación estándar e están incluídos no paquete `apache2-data`. Hai moitos outros módulos que se distribúen en paquetes separados, que en *debian* chámanse `libapache2-mod-*`:

```
# apt-cache search libapache2-mod
libapache2-mod-auth-ntlm-winbind - apache2 module for NTLM authentication
against Winbind
libapache2-mod-upload-progress - upload progress support for the Apache web
server
libapache2-mod-xforward - Apache module implements redirection based on X-
Forward response header
```

...

Uso de módulos

Se imos ao directorio onde están os módulos Apache dispoñibles `/etc/apache2/mods-available` e facemos unha lista, atoparemos ficheiros `*.load` e `*.conf`.

Os ficheiros cunha extensión `load` normalmente inclúen unha liña coa directiva `LoadModule`, por exemplo:

```
# cat userdir.load
LoadModule userdir_module /usr/lib/apache2/modules/mod_userdir.so
```

Ademais de cargar o módulo, en moitos casos é necesario realizar algunha configuración a través de directivas, polo que neses casos hai un ficheiro con `.conf`.

Se queremos que Apache use algún módulo, o que teríamos que facer é unha ligazón simbólica do ficheiro de extensión `.load` (e `.conf` se existe) no directorio `/etc/apache2/mods-enabled`. Podemos facer esta ligazón coa instrución **a2enmod**, por exemplo:

```
# a2enmod userdir
Enabling module userdir.
To activate the new configuration, you need to run:
  systemctl restart apache2
```

Para desactivala (eliminar a ligazón simbólica) empregamos a instrución **a2dismod**. despois de usar estes comandos tes que reiniciar o servizo.

Módulos activos por defecto

Para ver os módulos activados en apache2:

```
# apache2ctl -M

Loaded Modules:
  core_module (static)
  so_module (static)
  watchdog_module (static)
  http_module (static)
  log_config_module (static)
  ...
```

Módulo UserDir

En sistemas con varios usuarios, cada usuario pode ter un sitio web *no seu* directorio *home* usando o módulo UserDir. Os visitantes dunha URL

`http://example.com/~username/` recibirán o contido do directorio persoal do usuario "nome de usuario", no subdirectorio especificado pola directiva UserDir.

A directiva `UserDir` podemos modificala no ficheiro

`/etc/apache2/mods-available/userdir.conf` e pódese configurar de diferentes xeitos:

- `UserDir public_html`: Valor predeterminado, a URL `http://example.com/~rbowen/file.html` traducirase ao path do ficheiro `/home/rbowen/public_html/file.html`.
- `UserDir /var/html`: A URL `http://example.com/~rbowen/file.html` traducirase ao path do ficheiro `/var/html/rbowen/file.html`.
- `UserDir /var/www/*/docs`: A URL `http://example.com/~rbowen/file.html` traducirase ao path do ficheiro `/var/www/rbowen/docs/file.html`
- `UserDir public_html /var/html`: Para a URL `http://example.com/~rbowen/file.html`, Apache buscará `~rbowen`. Se non o atopa, Apache buscará `rbowen` en `/var/html`.

Desactivando userdir para algúns usuarios

Por exemplo, coa directiva `UserDir disabled root` desactivamos a funcionalidade que ofrece o módulo para o usuario `root`.

Configurando o directorio de acceso de cada usuario

Como podemos ver no ficheiro, `/etc/apache2/mods-available/userdir.conf` debemos configurar as opcións do directorio ao que se accede ao solicitar a páxina do usuario.

```
<Directory /home/*/public_html>
    AllowOverride FileInfo AuthConfig Limit Indexes
    Options MultiViews Indexes SymLinksIfOwnerMatch IncludesNoExec
    Require method GET POST OPTIONS
</Directory>
```

Activación do módulo

Para activar o módulo:

```
# a2enmod userdir
```

Reiniciamos o servidor, creamos unha carpeta `public_html` no home do usuario e creamos un ficheiro `index.html`. E podemos probar o acceso á url

`http://www.pagina1.org/~usuario`.

WebDAV

WebDAV (" *Edición e versións distribuídas na web* ") é un protocolo para facer de www un medio lexible e editable. Este protocolo proporciona funcionalidades para crear, cambiar e mover documentos nun servidor remoto (normalmente un servidor web). Isto úsase principalmente para permitir a edición de documentos servidos por un servidor web, pero tamén se pode aplicar a sistemas de almacenamento baseados na web, aos que se pode acceder desde calquera lugar. A maioría dos sistemas operativos modernos son compatibles con WebDAV, facendo que os ficheiros nun servidor WebDAV parezan almacenados nun directorio local.

Configuración dun servidor WebDAV

Para crear un directorio no noso servidor web que poida ser accesible a través dun cliente WebDAV debemos activar os módulos `dav` e `dav_fs`.

```
# a2enmod dav dav_fs
```

O primeiro é indicar o nome da base de datos de lock que se empregará, a través da directiva `DAVLockDB`. É importante ter especial coidado con esta directiva, xa que é unha fonte frecuente de erros.

```
DavLockDB /tmp/DAVLockDB
```

O que indica a directiva non é o nome dun ficheiro nin o dun cartafol, senón a parte inicial do nome dun ficheiro. O módulo creará un ficheiro cun nome `DAVLockDB.orig` e outro cun nome `DAVLockDB.xxxxxx` dentro do cartafol indicado, para o que é necesario que o usuario "*Apache*" teña permisos de escritura nel.

```
chown -R www-data:www-data /var/www
```

A continuación creamos unha sección `Directory` para o directorio ao que queremos acceder por WebDav e activamos o modo WebDav coa directiva `dav on`. Ademais, por seguridade, o acceso debe ser autenticado, polo que sería así:

```
DavLockDB /var/www/DavLock
Alias /webdav /var/www/webdav
<Directory /var/www/webdav>
    dav on
    AuthUserFile "/etc/apache2/claves/passwd.txt"
    AuthName "Palabra de paso"
    AuthType Basic
    Require valid-user
    Options Indexes FollowSymLinks MultiViews
    AllowOverride None
</Directory>
```

Finalmente podemos comprobar o acceso ao servidor WebDAV cun cliente.

Seguridade HTTPS

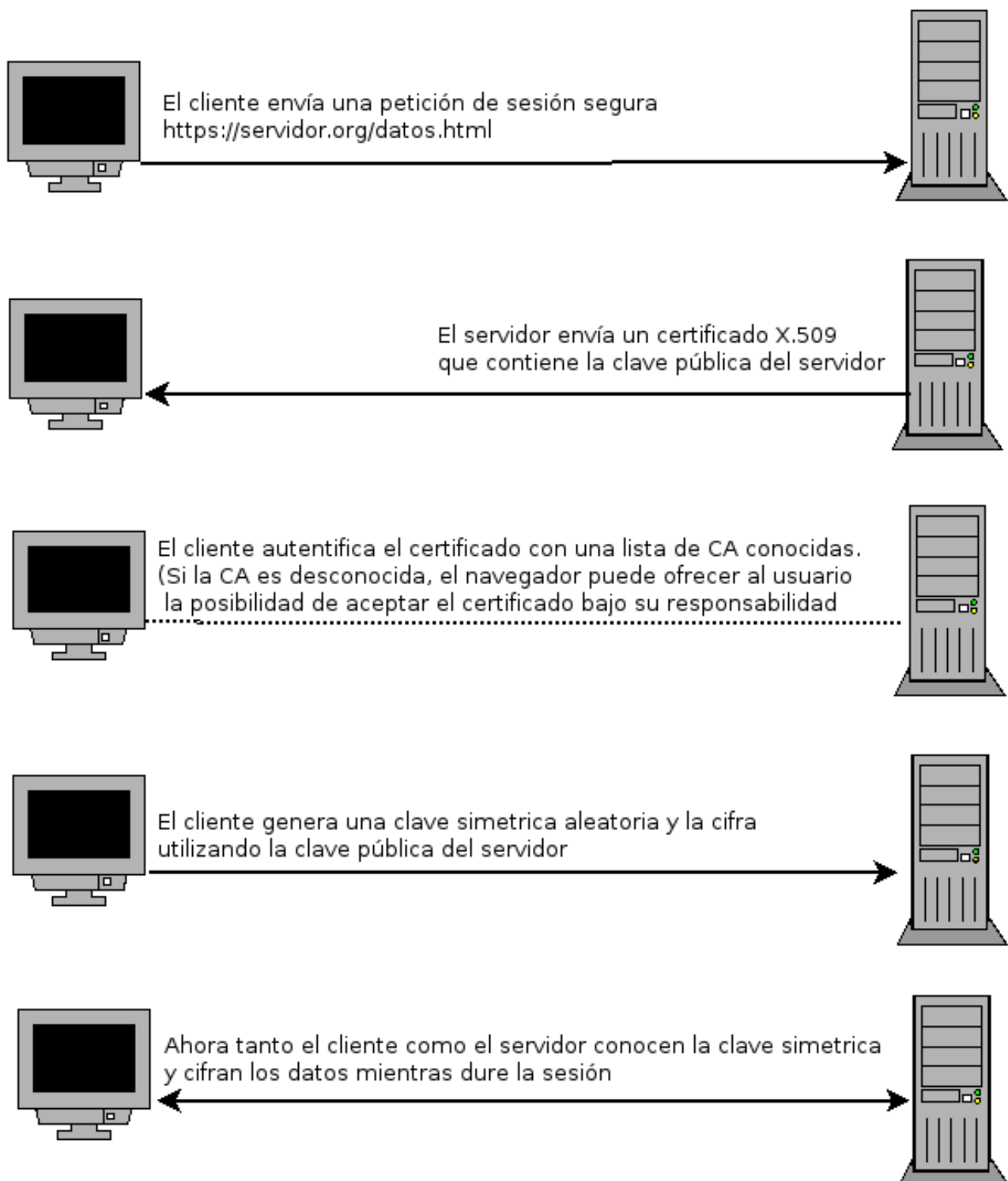
Introdución a HTTPS

Cada vez é máis necesario cifrar o contido que se transmite entre o cliente e o servidor, este proceso permítenos asegurar, por exemplo, o proceso de autenticación do usuario para evitar que alguén poida capturar un contrasinal de usuario e acceder a el de forma fraudulenta.

O cifrado da comunicación entre o navegador e o servidor web faise mediante o protocolo HTTPS, que ten as seguintes características principais:

- Utiliza o protocolo SSL (actualmente TLS) para o cifrado de datos.
- O servidor usa o porto 443 / tcp por defecto.
- Utiliza mecanismos de cifrado de chave pública e as claves públicas chámanse certificados.
- O formato dos certificados está especificado pola norma X.509 e normalmente son emitidos por unha entidade chamada **Autoridade de Certificación** (CA).
- No caso de HTTPS, a función principal da CA é demostrar a autenticidade do servidor e que pertence lexitimamente á persoa ou organización que o usa. Dependendo dos criterios empregados para verificar a autenticidade do servidor, emítense diferentes tipos de certificados X.509 (actualmente *úsase* o chamado *Certificado de validación estendido*).
- O navegador contén unha lista de certificados de CA nos que confía e inicialmente só acepta os certificados de servidor emitidos por unha destas CA.
- Unha vez aceptado o certificado dun servidor web, o navegador úsao para cifrar os datos que quere enviar ao servidor mediante o protocolo HTTPS e cando chega ao servidor, só pode descifralo, xa que é o único que ten a clave privada que a descifra.

O funcionamento de HTTPS dun xeito esquemático podería resumirse no seguinte gráfico:



Na seguinte unidade usaremos CA CaCert para obter un certificado X.509 para o noso servidor.

Hai outras CA que nos proporcionan certificados e non son gratuítos. Estas CA terán métodos máis robustos para validar o servidor, persoa ou organismo que solicita o certificado. E, polo tanto, poden dar un maior nivel de confianza.

Obtén un certificado en CAcert

Aínda que nos últimos tempos Let's Encrypt , que é unha entidade certificadora que ofrece certificados X.509 gratuítos para o cifrado de seguridade da capa de transporte (TLS), está de moda , a utilización dun cliente chamado Certbot que automatiza todo o ciclo de vida da xestión de certificados: obtención, renovación , revogación, instalación no servidor web, ... dificultáanos o estudo en profundidade do proceso de xestión dos nosos propios certificados.

Polo tanto, imos utilizar a CA chamada CAcert que nos permite levar a cabo un proceso similar ao que debemos facer se xestionamos os nosos certificados con algunha CA comercial.

O lema de **CAcert** é *Certificados dixitais gratuítos para todos* e é que o uso de certificados emitidos por CA comerciais non é posible para todos os sitios de Internet debido ao seu custo, o que limita o seu uso a transaccións económicas ou sitios con datos relevantes. CAcert é unha organización sen ánimo de lucro que mantén unha infraestrutura equivalente a unha CA comercial aínda que con certas limitacións.

Os pasos a seguir para usar un certificado X.509 emitido por CAcert son os seguintes:

- Rexístrate como usuario na páxina web.
- Rexistra o dominio para o que queremos obter o certificado. (opción **Dominios -> Engadir**)
- CAcert verifica que podemos facer un uso lexítimo do dominio enviando unha mensaxe de correo electrónico.
- Rexistra un certificado de servidor mediante unha solicitude de sinatura de certificado (CSR).
- Configure o servidor web co certificado X.509 emitido pola CA.

Instalación do certificado raíz de CAcert

Unha das limitacións do uso desta CA é que o seu certificado raíz non está instalado de xeito predeterminado nos navegadores, polo tanto temos que instalalo manualmente. Para iso, simplemente accedemos ao sitio web de CAcert e na opción de **certificado raíz** podemos descargar a versión do **certificado raíz (formato PEM)** . A continuación marcamos a opción: *Confíe nesta CA para identificar sitios web* e xa o temos instalado.

Este proceso pode axudarnos a comprender a circunstancia de que unha empresa ten a súa propia CA para xestionar os seus propios certificados e, por exemplo, os seus empregados teñen que instalar o certificado raíz dun xeito similar.

Creación da RSE

CSR significa "Certificate Signing Request" ou " *Solicitude de sinatura de certificado* " . Primeiro de todo necesitamos xerar unha clave privada RSA de 4096 bits usando a instrución:

```
# openssl genrsa 4096 > /etc/ssl/private/ssl-cert.key
```

Modificamos adecuadamente os propietarios e permisos:

```
# chown root:ssl-cert /etc/ssl/private/ssl-cert.key
# chmod 640 /etc/ssl/private/ssl-cert.key
```

Usando a clave privada, xeramos unha CSR a través da instrución:

```
# openssl req -new -key /etc/ssl/private/ssl-cert.key -out
/etc/ssl/private/midominio.csr
```

A CSR contén información que finalmente se incluírá no certificado SSL, como o seu nome ou o da empresa, o enderezo, o país de residencia ou o *common name* (dominio para o que se xera o SSL), ademais destes datos tamén incluírá unha clave pública que tamén se incluírá no seu certificado.

Este ficheiro csr é o que enviamos á CA a través do formulario web (**Certificados de servidor -> Nova** opción), que o procesa e obtén del un certificado X.509 compatible coa nosa clave privada, pero emitido pola CA.

O certificado que descargamos gárdase no ficheiro
/etc/ssl/certs/midominio-cacert.pem.

Configuración HTTPS

Imos configurar o acceso co protocolo HTTPS a `informatica.manuelsanchez.org`.

O primeiro que temos que facer é activar o módulo SSL:

```
# a2enmod ssl
```

A continuación, imos crear un host virtual para o noso nome de dominio completo a partir do ficheiro predeterminado para a configuración HTTPS, no directorio /etc/apache2/sites-available executamos:

```
# cp default-ssl.conf informatica-ssl.conf
```

E configurámolo adecuadamente:

```
...
ServerName  informatica.manuelsanchez.org
DocumentRoot /var/www/informatica
...
SSLEngine on
SSLCertificateFile /etc/ssl/certs/informatica-cacert.pem
SSLCertificateKeyFile /etc/ssl/private/ssl-cert.key
SSLCertificateChainFile /etc/ssl/certs/DigiCertCA.crt
...
```

Coa directiva `SSLEngine` activamos o uso de HTTPS, `SSLCertificateFile` permítenos indicar o certificado emitido pola CA e con `SSLCertificateKeyFile` indicar a nosa clave privada.

Finalmente activamos o sitio:

```
# a2ensite prueba-ssl.conf
```

Redirixindo o tráfico HTTP a HTTPS

Podemos facer unha redirección para que cando accedamos con HTTP, o recurso se solicite mediante HTTPS. Para iso, no ficheiro de configuración do host virtual `/etc/apache2/sites-available/informatica.conf` podemos incluír un `redirect`:

```
...
redirect premanent / https://informatica.manuelsanchez.org
...
```

Consellos de seguridade

Ademais de usar HTTPS podemos ter en conta algúns outros consellos de seguridade:

Ocultar versión e sistema

No ficheiro `/etc/apache2/conf-enabled/security.conf` podemos configurar as directivas `ServerSignature` e `ServerTokens`.

- `ServerTokens`: Onde configuramos a información devolta nas cabeceiras de resposta HTTP. Ademais, se a outra política está activa, mostra esa información nas páxinas de erro predeterminadas.
- `ServerSignature`: Permite controlar se a información aparece nas páxinas de erro de xeito predeterminado (a versión de apache que instalamos, a IP e o porto).

Nun servidor de produción os valores deben ser:

```
ServerSignature Off
ServerTokens Prod
```

Desactivar a listaxe de directorios

É recomendable desactivar a opción `Indexes` para evitar que apache2 mostre a lista de ficheiros e directorios se non atopa un ficheiro que teña un nome predeterminado pola directiva `DirectoryIndex`. Por exemplo:

```
<Directory /var/www/pagina1/>
    Options -Indexes
</Directory>
```

Desactivar módulos innecesarios

Isto é necesario por dúas razóns, para reducir a ocupación de recursos, aumentar o rendemento e evitar posibles ataques debido ás vulnerabilidades que poden ter algúns módulos. Podemos visualizar os módulos activos executando:

```
# apache2ctl -M
```

Por exemplo, por defecto temos activo o módulo `status` que nos permite ver o estado do servidor se accedemos á URL `/server-status` (por defecto só desde `localhost`). Poderíamos desactivar este módulo:

```
# a2dismod status
```

Desactivar as ligazóns simbólicas

Por defecto permítelle seguir as ligazóns simbólicas dentro do noso servidor virtual. Esta funcionalidade pode ter consecuencias indesexables se, debido a unha mala configuración, se filtra o contido do ficheiro que non debería ser visible para o servidor web. Polo tanto, no ficheiro `/etc/apache2/apache2.conf` deberíamos ter:

```
<Directory /var/www/>
  # Options Indexes FollowSymLinks
  AllowOverride None
  Require all granted
</Directory>
```

Comentamos ou eliminamos a liña que permite mostrar o índice de ficheiros e directorios e seguir ligazóns simbólicas.

Manteñámonos actualizados

Para rematar, é moi importante manter o servidor web actualizado cos parches de seguridade que se publican no noso sistema, para iso:

```
# apt-get update
# apt-get upgrade

# apache2ctl -v
Server version: Apache/2.4.25 (Debian)
Server built: 2017-09-19T18:58:57
```

Módulo Apache2, PHP e MySQL

Instalamos apache2 e o módulo que permite que os procesos apache2 poidan executar o código PHP:

```
apt install libapache2-mod-php
```

Cando facemos a instalación, hai que reiniciar o servizo de Apache

Configuración de PHP

Se observas a configuración de php para apache2:

- `/etc/php/7.0/apache2/conf.d`: Módulos instalados nesta configuración PHP (ligazóns simbólicas a `/etc/php/7.0/mods-available`).
- `/etc/php/7.0/apache2/php.ini`: Configuración de PHP para este escenario.

Comprobación de PHP

Podemos crear unha páxina web no noso document root con extensión php e que conteña:

```
<?php
    phpinfo();
?>
```

Configuración de MySQL

Para instalar o cliente de MySQL

```
apt install default-mysql-client
```

Para instalar o servidor de MySQL

```
apt install mariadb-server
```

asegurarse de ter comentados:

```
#skip-networking
#bind-address = <some ip-address>
```

Para instalar o módulo php-mysql

```
apt-get install php7.3-mysql
```

Crear un usuario en mysql:

```
create user 'nome'@'%' identified by 'password';
grant all privileges on *.* to 'nome'@'%';
```

conexión co cliente mysql:

```
mysql -u usuario -p -h ip_servidor nome_base_datos
```

Comprobación de MySQL

Podemos crear unha páxina .php co seguinte código: (cambiando os valores de ip, nome de usuario, password e base de datos:

```
<?php
/* Host name of the MySQL server */
$host = 'localhost';
/* MySQL account username */
$user = 'myUser';
/* MySQL account password */
$password = 'myPasswd';
/* The schema you want to use */
$database = 'mySchema';
/* Connection with MySQLi, procedural-style */
$mysqli = mysqli_connect($host, $user, $password, $database);
/* Check if the connection succeeded */
if (!$mysqli)
{
    echo 'Connection failed<br>';
    echo 'Error number: ' . mysqli_connect_errno() . '<br>';
    echo 'Error message: ' . mysqli_connect_error() . '<br>';
    die();
}
echo 'Successfully connected!<br>';
?>
```